

Documentação Técnica — Geração e Edição da Folha de Frequência

Continuação do Sistema de Grade — FATEC Zona Leste

Documento técnico-base para o desenvolvimento do módulo de Folha de Frequência. Ele consolida a Especificação funcional e o prompt de requisitos do projeto, e organiza, valida e documenta: escopo, fluxo, modelo DOCX oficial, regras de conversão de horários, Calendário Acadêmico, etapa de conferência/edição, arquitetura Java + Spring Boot, rotas da API, estratégia de testes e backlog de issues. **Este documento não implementa o sistema:** ele serve de referência para que um Tech Lead ou desenvolvedor sênior comece a implementação com segurança.

Como ler este documento. Os pontos ainda não definidos aparecem como **DECISÃO A CONFIRMAR** seguido de um código (D-01 a D-16). Todos estão reunidos, com a contradição encontrada, o impacto e a recomendação, na **seção 18**. Nenhum requisito novo foi inventado: onde as fontes divergem, a divergência é apontada em vez de ser resolvida silenciosamente.

Sumário

1. Objetivo e escopo do sistema	2
2. Fluxo principal do sistema	3
3. Modelo DOCX oficial e placeholders	4
4. Dados do professor e campos adicionais	6
5. Seleção de professor e grade	6
6. Grade Horária: estrutura e conversão de horários	7
7. Folha de Frequência: datas e formatação	8
8. Calendário Acadêmico	9
9. Etapa de conferência e edição	11
10. O que o sistema não implementa	12
11. Arquitetura técnica	12
12. Rotas da API	15
13. Estrutura de pastas e templates	18
14. Estratégia de testes	18
15. Backlog e Issues	19
16. Checklist do escopo	20
17. Regra fundamental e resultado esperado	20
18. Decisões pendentes e contradições	21
Anexo A — Issues detalhadas	23

1. Objetivo e escopo do sistema

1.1 Visão geral

O módulo de Folha de Frequência faz parte do sistema desenvolvido para a FATEC Zona Leste e dá continuação ao Sistema de Grade. Ele automatiza a preparação de um documento oficial usado pelos professores, reaproveitando informações que já existem no banco da instituição: professores, disciplinas, cursos, turmas, grades horárias, aulas, horários, períodos e semestres/anos.

1.2 Objetivo

O sistema permite que o usuário selecione um professor, a grade horária correspondente e informe mês e ano desejados. A partir daí, utiliza os dados já existentes para gerar/preencher automaticamente a Folha de Frequência em DOCX, no modelo oficial da FATEC — atualizando datas, dias da semana, grade horária e informações do calendário acadêmico. O usuário confere e edita o documento antes de gerar/salvar o DOCX final.

Para isso, o sistema deve:

- buscar os dados do professor e permitir selecionar o professor e a grade correspondente;
- permitir selecionar mês e ano;
- interpretar os horários da grade e converter os horários reais para os slots (blocos) do modelo oficial;
- preencher a Grade Horária, as datas, os dias da semana e o título mês/ano;
- identificar domingos e aplicar o sombreamento correspondente;
- considerar dias sem aula definidos pelo Calendário Acadêmico (baixa prioridade — seção 8);
- preservar o modelo oficial e gerar sempre uma **cópia** preenchida;
- disponibilizar o documento para conferência e permitir ajustes antes da versão final.

1.3 Conceito: sistema de preparação, não de controle

Este projeto NÃO é um sistema eletrônico de controle de ponto. Ele não registra entrada, saída, horário efetivamente trabalhado, presença, falta, atraso ou justificativa. O objetivo é automatizar a **preparação** da Folha de Frequência, e não controlar a frequência do professor.

O documento gerado é conferido pelo usuário. A responsabilidade pelo preenchimento e pelo controle oficial da frequência permanece no processo institucional correspondente. Por isso, nenhuma funcionalidade de controle de presença deve ser criada sem solicitação explícita.

O sistema não é responsável por:

- registrar presença ou falta;
- controlar ponto, entrada ou saída;
- controlar atraso ou registrar justificativas;
- controlar reposição de aula;
- substituir o sistema oficial de ponto da instituição.

1.4 Escopo

Dentro do escopo	Fora do escopo neste momento
• Seleção de professor, grade, mês e ano • Consulta dos dados do professor e da grade • Conversão dos horários para os slots do modelo • Preenchimento da Grade Horária (SIGLA) • Datas, dias da semana e título mês/ano • Sombreamento de domingos e dias sem aula • Meses com 28, 29, 30 e 31 dias • Consulta ao Calendário Acadêmico (baixa prioridade) • Geração do DOCX a partir do modelo oficial • Conferência e edição antes da versão final • Geração/salvamento do DOCX final	• Controle eletrônico de ponto • Entrada, saída, atraso, falta e justificativas • Reposição automática e substituição de professor • Controle de assinatura • Sistema administrativo completo do Calendário Acadêmico • Novo banco de dados ou cadastro paralelo de professores/grades • Login/autenticação (não definido — DECISÃO A CONFIRMAR (D-16)) • Qualquer funcionalidade não confirmada pelos responsáveis

1.5 Premissas

- O banco de dados e as estruturas do Sistema de Grade **já existem** e serão reutilizados; nada é duplicado.
- O modelo DOCX oficial é fornecido pela instituição e tratado como **template somente leitura**.
- Backend em Java 21 + Spring Boot; a documentação e a interface são em português.
- A implementação existente da Grade (entidades, regra de horários) deve ser analisada antes de qualquer alteração de regra.

2. Fluxo principal do sistema

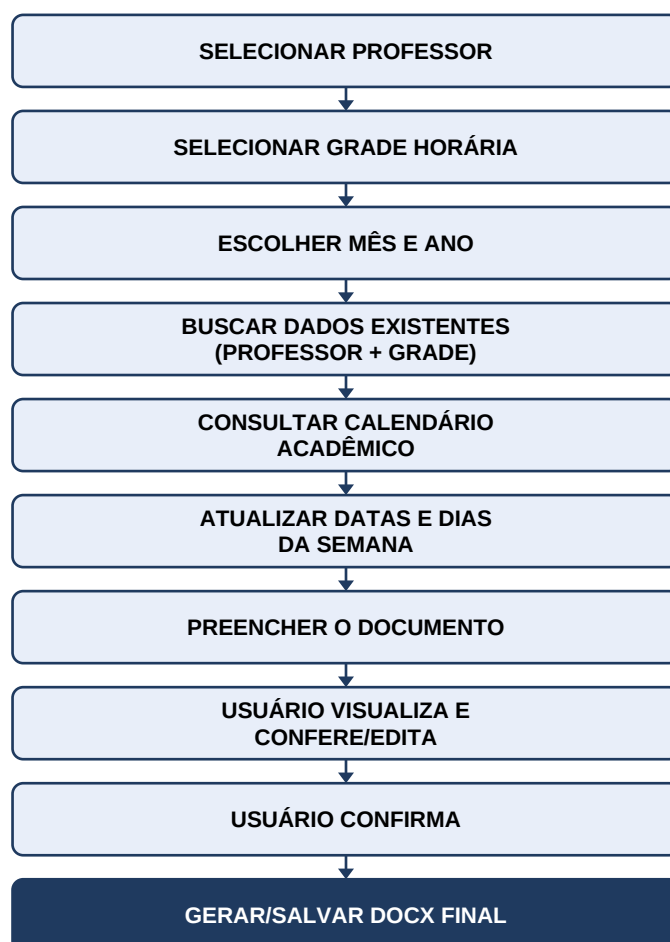


Figura 1 — Fluxo principal do sistema

A etapa de conferência/edição é obrigatória antes da geração do documento final (seção 9). O fluxo detalhado, com o componente responsável por cada passo, é:

#	Ator / componente	Passo
1	Usuário	Acessa o sistema e seleciona um professor.
2	ProfessorService	Busca os dados do professor (nome, código, matrícula, regime, categoria).
3	GradeService	Apresenta as grades disponíveis do professor (ex.: "Grade 2026/2").
4	Usuário	Seleciona a grade e informa mês e ano.
5	GradeService	Recupera as aulas da grade: dias, horários, período, disciplina (SIGLA), turma, curso.
6	GradeService	Converte os horários reais para os slots do modelo oficial (seção 6).
7	FolhaFrequenciaService	Calcula as datas do mês, o dia da semana de cada data e identifica os domingos.
8	CalendarioService	Consulta feriados, emendas, recessos e demais dias sem aula (seção 8).
9	FolhaFrequenciaService	Consolida tudo em um modelo de dados da folha (<i>FolhaModel</i>) e reúne avisos.
10	FolhaFrequenciaDocxService	Copia o template, substitui placeholders, preenche a Grade Horária, as datas, os dias da semana, aplica sombreamento e limpa linhas inexistentes.
11	Usuário	Visualiza, confere e, se necessário, edita o documento preenchido.
12	Usuário	Confirma.
13	FolhaFrequenciaDocxService	Gera o DOCX final a partir dos dados confirmados.

3. Modelo DOCX oficial e placeholders

3.1 Princípios

O sistema utiliza o modelo oficial da Folha de Frequência fornecido pela instituição. O modelo é um **template**: nunca é sobrescrito. A aplicação altera somente os dados necessários, sem modificar fonte, tamanho, margens, espaçamento, bordas, largura de tabelas, altura de linhas, cabeçalho, rodapé ou estrutura das páginas.

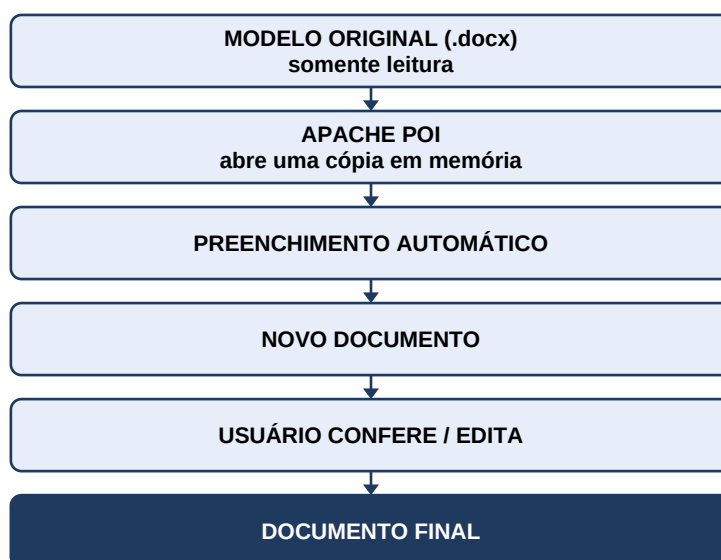


Figura 2 — O template nunca é alterado; todo o trabalho ocorre sobre uma cópia

3.2 Placeholders definidos no modelo

Placeholder	Campo	Origem do dado	Onde é preenchido	Service	Obrig.
{{PROFESSOR}}	Nome completo do professor	PROFESSOR.nome	Bloco de identificação	ProfessorService	Sim
{{COD}}	Código relacionado ao professor (se existir)	PROFESSOR.codigo	Bloco de identificação	ProfessorService	Não (D-10)
{{MAT}}	Matrícula do professor	PROFESSOR.matricula	Bloco de identificação	ProfessorService	Não (D-10)
{{RJ}}	Regime jurídico do professor	PROFESSOR.regime_contrato	Bloco de identificação	ProfessorService	Não (D-10)
{{MES}}	Mês selecionado (numérico, 2 dígitos)	Seleção do usuário	Título da folha	FolhaFrequenciaService	Sim
{{ANO}}	Ano selecionado	Seleção do usuário	Título da folha	FolhaFrequenciaService	Sim

O título da folha segue o padrão **FOLHA DE FREQUÊNCIA – {{MES}} / {{ANO}}** (ex.: “FOLHA DE FREQUÊNCIA – 09 / 2026”). Os placeholders são substituídos mantendo a formatação original. A posição exata de cada um no arquivo depende da inspeção do modelo (**DECISÃO A CONFIRMAR (D-12)**).

3.3 Placeholders divididos em vários “runs”

No DOCX, o Word pode dividir internamente um texto como `{{PROFESSOR}}` em vários trechos (*runs*), por exemplo “{”, “PROFESSOR” e “}”. Uma busca simples por string, run a run, **não encontraria** o placeholder. A implementação deve, portanto, tratar o parágrafo como um todo:

- 1 percorrer todos os parágrafos do corpo, das células de tabelas (inclusive tabelas aninhadas), cabeçalhos e rodapés;
- 2 concatenar o texto dos runs do parágrafo e localizar o placeholder por expressão regular;
- 3 gravar o valor no primeiro run envolvido (o que contém “{”), preservando a formatação dele;
- 4 esvaziar o texto dos demais runs que compunham o placeholder, sem removê-los;
- 5 repetir até não restarem ocorrências no parágrafo.

Observação: caixas de texto e formas flutuantes não são cobertas pela API de parágrafos (*XWPFParagraph*); se o modelo as usar, será necessário um tratamento específico (D-12).

3.4 Áreas do modelo e o que pode ser alterado

Área do modelo	Conteúdo	Alterável pelo sistema?
Identificação do professor	Nome, código, matrícula, regime jurídico	Sim — apenas os valores dos placeholders
Título da folha	FOLHA DE FREQUÊNCIA – MM / AAAA	Sim — {{MES}} e {{ANO}}
Grade Horária	Dias (segunda a sábado) × slots 1º a 6º por turno	Sim — apenas o conteúdo das células (SIGLA)
Folha de Frequência	Até 31 linhas: número do dia, dia da semana, demais colunas	Sim — número do dia, dia da semana e sombreamento; demais colunas não
Assinaturas e demais textos oficiais	Campos de assinatura e informações fixas	Não
Cabeçalho, rodapé, margens, fontes, bordas	Estrutura visual oficial	Não

DECISÃO A CONFIRMAR (D-12) — Estrutura interna do DOCX ainda não inspecionada. A tabela acima reflete a Especificação. Antes de implementar, é preciso abrir o arquivo oficial e produzir um *mapa do template*: quantidade e ordem das tabelas, linhas e colunas de cada uma, células com placeholders, células que recebem sombreamento e elementos que não podem ser tocados. Recomenda-se concentrar esses índices em uma única classe de configuração (ex.: *TemplateLayout*) — ou, melhor, localizar áreas por marcadores no próprio template — para que uma mudança no modelo afete um só lugar.

4. Dados do professor e campos adicionais

O sistema usa os dados já existentes no banco atual — **não é criado um banco separado** nem um segundo cadastro de professores. Exemplo do bloco de identificação: *PROFESSOR: Alexandre Falcão Sanseverino | CÓDIGO: — | MATRÍCULA: — | REGIME: Determinado | CATEGORIA: Professor.*

Campo do documento	Placeholder	Fonte no banco	Observação
Nome	{{PROFESSOR}}	PROFESSOR.nome	Nome completo.
Código	{{COD}}	PROFESSOR.codigo	Pode não existir; ver D-10.
Matrícula	{{MAT}}	PROFESSOR.matricula	Pode estar vazia (exemplo mostra “—”); ver D-10.
Regime jurídico	{{RJ}}	PROFESSOR.regime_contrato	Ex.: “Determinado”. Confirmar se o texto do banco é o mesmo do documento.
Categoria	não há placeholder	a confirmar	Aparece no exemplo e no ProfessorService, mas o modelo não lista {{CATEGORIA}} (D-09).

Regra para campos ausentes: primeiro verificar se o campo já existe; se existir, usar; se não existir, adicionar apenas o campo que falta à estrutura existente, sem criar várias estruturas novas. Sempre que algo exigido pelo documento não estiver no banco, registrar: (1) qual informação; (2) por que é necessária; (3) justificativa; (4) onde deve ser armazenada; (5) impacto.

Caso conhecido	Campo “categoria”
Identificação / explicação	Usada no exemplo de identificação e prevista no ProfessorService; não há placeholder correspondente no modelo.
Justificativa	Só se justifica se o modelo oficial exibir a categoria (D-09).
Onde armazenar	Na entidade/tabela de professor já existente — uma coluna, sem nova tabela.
Impacto	Alteração de esquema no banco atual, ajuste em Entity/DTO e migração dos dados existentes.

5. Seleção de professor e grade

O usuário seleciona o professor (ex.: “Alexandre Falcão Sanseverino”) e, em seguida, a grade correspondente (ex.: “Grade 2026/2”). A grade selecionada é a **fonte** dos horários: a Folha não cria uma cópia independente das informações da Grade — ela lê a Grade existente no momento da geração.

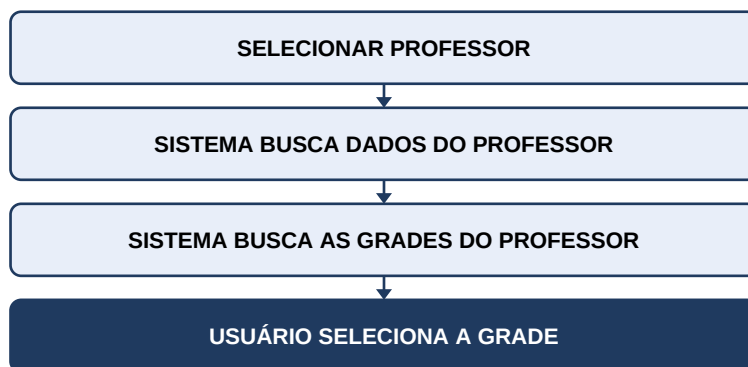


Figura 3 — Sequência de seleção

A grade fornece: dia da semana, horário inicial e final, período, disciplina (código, SIGLA e nome), turma e curso. A disciplina possui código completo e SIGLA separada:

Código	SIGLA	Nome	Uso no documento
MAG004	MAG	Álgebra Linear	Grade Horária do documento usa MAG

O banco mantém o código completo; na Grade Horária do documento utiliza-se **sempre a SIGLA** (ex.: MAG, DW, ESP) — nunca o código completo.

Validações da seleção

- o professor existe; a grade existe e **pertence** ao professor informado;
- mês entre 1 e 12; ano válido (faixa a definir);
- mês/ano dentro do período da grade (ex.: “Grade 2026/2” e mês 03/2026): avisar ou bloquear? **DECISÃO A CONFIRMAR (D-13)**.

6. Grade Horária: estrutura e conversão de horários

6.1 Slots fixos do modelo oficial

Turno	1º	2º	3º	4º	5º	6º
Manhã	07:30	08:20	09:20	10:10	11:10	12:00
Tarde	13:00	13:50	14:50	15:40	16:40	17:30
Noite	19:20	20:10	21:10	22:00	—	—

Dias da semana: segunda a sábado. O **domingo não recebe aulas** na grade (o modelo não possui linha específica para aula aos domingos).

6.2 Conversão do horário real para o slot do modelo

O horário real cadastrado na grade (ex.: 08:00 – 08:50) nem sempre coincide com os horários do modelo. A regra definida é: **converter o horário inicial para o slot mais próximo; em caso de empate entre dois slots, usar o próximo (o posterior)**. A regra reproduz os quatro exemplos da Especificação:

Horário real	Slots vizinhos (distância)	Resultado	Motivo
08:00 – 08:50	07:30 (30 min) · 08:20 (20 min)	2º (08:20)	08:20 é o mais próximo
08:50 – 09:40	08:20 (30 min) · 09:20 (30 min)	3º (09:20)	Empate → próximo slot
09:50 – 10:40	09:20 (30 min) · 10:10 (20 min)	4º (10:10)	10:10 é o mais próximo
10:40 – 11:30	10:10 (30 min) · 11:10 (30 min)	5º (11:10)	Empate → próximo slot

Pseudocódigo da regra (a lista de slots fica em uma única enumeração, nunca espalhada pelo código):

```
SlotModelo converter(LocalTime inicio):
    melhor = null
    para cada slot em SlotModelo.todosEmOrdemCrescente(): // 07:30 ... 22:00
        dist = abs(minutos(inicio) - minutos(slot.horario))
        se melhor == null OU dist <= melhor.dist: // "<=": empate fica com o POSTERIOR
            melhor = (slot, dist)
    retornar melhor.slot // turno e ordinal (10..60) derivam do slot
```

Onde fica: essa lógica não pode estar no Controller. Fica no *GradeService*, delegando para uma classe de domínio/serviço específica (ex.: *ConversorHorarioSlot* + enum *SlotModelo*), o que a torna testável isoladamente.

DECISÃO A CONFIRMAR (D-07) — Pontos que a regra ainda não cobre: (a) tolerância máxima de distância (um horário muito distante de qualquer slot deve ser convertido ou gerar aviso?); (b) aulas com mais de 50 minutos ou geminadas (ocupam um ou vários slots?); (c) conferir a regra contra a implementação já existente no Sistema de Grade antes de qualquer alteração.

6.3 Preenchimento da Grade Horária

Para cada aula da grade, a SIGLA da disciplina é gravada na célula correspondente ao **dia da semana** e ao **slot** convertido. Exemplo pela regra: terça-feira, disciplina DW, aula das 08:00 às 08:50 → slot 2º da manhã:

Terça-feira — Manhã	1º	2º	3º	4º	5º	6º
Conteúdo		DW				

DECISÃO A CONFIRMAR (D-08) — Exemplo da Especificação x regra de conversão. O exemplo original mostra “DW” nos seis slots (1º ao 6º) de uma terça-feira, o que só ocorreria se houvesse seis aulas seguidas de DW naquele dia. Pela regra de conversão, cada aula preenche apenas o slot resultante. Confirmar se o exemplo é meramente ilustrativo. Recomendação: preencher somente os slots resultantes da conversão.

Conflitos: se duas aulas diferentes forem convertidas para o mesmo dia e slot, o comportamento (erro de validação, concatenar SIGLAS ou aviso) está indefinido — **DECISÃO A CONFIRMAR (D-14)**. Recomendação: rejeitar com erro descritivo (dia, slot e disciplinas envolvidas), pois o documento oficial comporta uma sigla por célula.

7. Folha de Frequência: datas e formatação

7.1 Cálculo das datas

A tabela é atualizada conforme o mês selecionado. O sistema atualiza automaticamente mês, ano, número do dia e dia da semana, calculando: quantidade de dias do mês, dia da semana de cada data e domingos. O título segue o padrão **FOLHA DE FREQUÊNCIA – 09 / 2026**.

- **Número do dia:** dois dígitos (01, 02, ... até o último dia do mês).
- **Dia da semana:** Segunda-feira, Terça-feira, Quarta-feira, Quinta-feira, Sexta-feira, Sábado, Domingo — obtido a partir da data e mapeado por uma tabela fixa no código (não depender do idioma/locale do servidor). A grafia deve ser idêntica à do modelo oficial.
- **Exemplo — setembro/2026:** 01 Terça-feira, 02 Quarta-feira ... 30 Quarta-feira; domingos nos dias 06, 13, 20 e 27.
- **Exemplo — fevereiro/2027:** 28 dias (01 é segunda-feira); domingos nos dias 07, 14, 21 e 28.

7.2 Meses com quantidades diferentes de dias

O modelo possui estrutura fixa de **31 linhas**. Quando o mês tem menos dias, as linhas dos dias inexistentes devem ser **limpas** — nenhuma informação antiga pode permanecer. Não se criam nem se removem linhas: a linha existente é esvaziada e fica sem sombreamento.

Mês	Dias	Linhas a limpar
Fevereiro (ano não bissexto, ex.: 2027)	28	29, 30 e 31
Fevereiro (ano bissexto, ex.: 2028)	29	30 e 31
Abril, junho, setembro e novembro	30	31
Janeiro, março, maio, julho, agosto, outubro e dezembro	31	nenhuma

7.3 Formatação das linhas

Situação da linha	Regra	Aparência
Dia inexistente no mês	Limpar conteúdo; não sombrear	Linha vazia, sem preenchimento
Domingo	Linha inteira sombreada	Cinza #D9D9D9
Dia sem aula (Calendário)	Alterar apenas a aparência da linha existente	Preenchimento definido pelo projeto — DECISÃO A CONFIRMAR (D-05)
Dia normal	Manter o preenchimento original	Original/branco

Precedência sugerida quando mais de uma regra se aplica à mesma linha: (1) dia inexistente; (2) domingo; (3) dia sem aula; (4) dia normal. Por exemplo, um feriado que caia em domingo continua cinza (**DECISÃO A CONFIRMAR (D-05)**).

Implementação: como todo documento parte de uma cópia limpa do modelo, o sombreadamento só precisa ser *aplicado* onde a regra exige (propriedade de sombreadamento *w:shd* com preenchimento D9D9D9 em todas as células da linha). As demais propriedades da linha — fonte, bordas, altura — não são tocadas.

8. Calendário Acadêmico

8.1 Finalidade e prioridade

O sistema usa o Calendário Acadêmico da FATEC para identificar datas em que normalmente haveria aula, mas que não devem ser tratadas como aula normal (feriados, emendas, recessos e outras datas indicadas no calendário).

Prioridade: BAIXA. Essa funcionalidade não deve bloquear o desenvolvimento principal da Folha: o fluxo completo (professor → grade → mês/ano → DOCX → conferência) funciona sem ela. A Especificação, porém, lista a consulta ao calendário no escopo obrigatório — **DECISÃO A CONFIRMAR (D-02)**.



Figura 4 — Regra entre Grade Horária e Calendário Acadêmico

Reposição não é implementada nesta versão: mesmo que o calendário traga uma “sugestão de reposição”, essa informação não gera uma nova aula na grade (sem lógica de reposição, troca de dia/horário, substituição ou compensação).

8.2 Estrutura de um evento do calendário

Campo	Exemplo	Descrição
data	07/09/2026 (ISO no JSON: 2026-09-07)	Data do evento
tipo	FERIADO	FERIADO, EMENDA, RECESSO, SEM_AULA, EVENTO_ACADÊMICO, OUTRO
descrição	Independência do Brasil	Texto livre
semAula	true	Campo mais importante: indica se a data tem ou não aula, independentemente do nome do tipo

DECISÃO A CONFIRMAR (D-04) — Semântica do campo semAula. O exemplo da Especificação mostra um FERIADO com “semAula = false (não há aula)”, mas o nome do campo sugere o contrário (true = sem aula). Recomendação: **semAula = true significa que NÃO há aula** e o exemplo deve ser corrigido para true; alternativa: renomear o campo para *temAula*. Este documento adota semAula = true.

8.3 Consumo pelo sistema

O *CalendarioService* expõe consultas simples e independentes do DOCX: obter o evento de uma data, listar os eventos de um mês e informar se uma data possui aula. A fonte dos dados fica atrás de uma interface, de modo que trocar a origem (arquivo → banco) não afete a geração do documento. Enquanto a issue do calendário não for implementada, a implementação devolve “nenhum evento”.

```
{
  "anoLetivo": 2026,
  "semestre": 2,
  "eventos": [
    { "data": "2026-09-07", "tipo": "FERIADO",
      "descricao": "Independência do Brasil", "semAula": true }
  ]
}
```

Exemplo completo. Professor com aula toda segunda-feira, disciplina DW. Ao gerar setembro/2026: 07/09/2026 é segunda-feira e a grade indica aula, mas o calendário marca a data como feriado (sem aula) — logo, 07/09/2026 não é tratado como aula normal. Já 14/09/2026, também segunda-feira com aula na grade e sem evento de “sem aula”, é considerado normalmente.

Escopo da regra. Pela Figura 4, o cruzamento só importa quando existe aula na grade naquela data. Não está definido se um feriado em dia sem aula do professor também deve alterar a linha — **DECISÃO A CONFIRMAR (D-06)**. Recomendação: seguir a Figura 4 (só dias com aula na grade), expondo no preview a lista completa de eventos do mês para conferência.

8.4 O que significa “dia sem aula” no documento

Quando uma data é identificada como dia sem aula, o sistema **não deve**: criar nova linha; registrar falta; registrar presença; colocar X ou ponto; registrar reposição. Deve apenas **alterar a aparência da linha existente**, conforme a regra definida:

Data	Aparência
Normal	Aparência original
Domingo	Cinza (#D9D9D9)
Dia sem aula	Preenchimento definido para dia sem aula — cor/tratamento ainda não definidos

DECISÃO A CONFIRMAR (D-05) / DECISÃO A CONFIRMAR (D-06) — Nem a Especificação nem o prompt definem a cor (ou texto) do dia sem aula, e a Grade Horária é semanal (a data 07/09 não altera a grade). É necessário confirmar com a FATEC o que “não tratar como aula” significa visualmente no DOCX. Recomendação: uma cor distinta do cinza dos domingos, sem inserir texto, até haver definição oficial.

8.5 Etapa futura: importação e revisão do calendário

A ideia de evolução (fora do escopo atual) é:

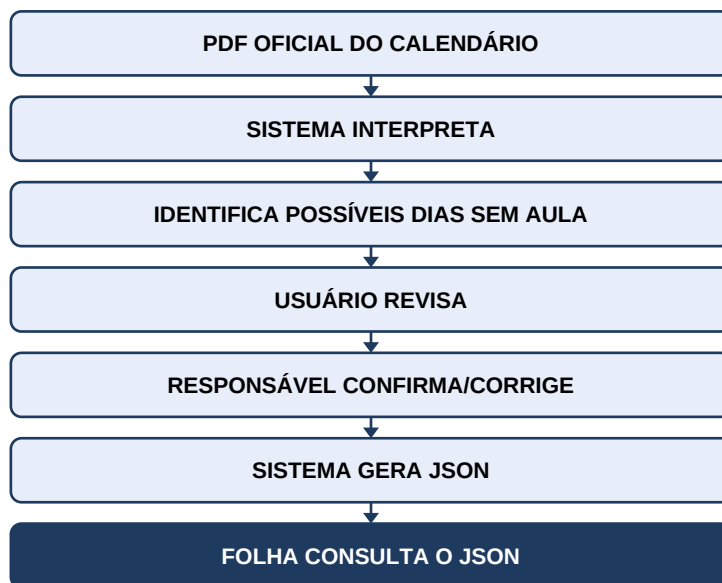


Figura 5 — Importação futura do calendário (baixa prioridade)

A futura tela de revisão deve permitir visualizar as datas identificadas e o motivo, corrigir, remover, adicionar e confirmar; após a confirmação, gera-se uma representação estruturada (inicialmente JSON). **Não** implementar agora um sistema administrativo completo de calendário.

DECISÃO A CONFIRMAR (D-03) — Fonte do calendário. A Especificação prevê “tela administrativa ou importação de CSV/Excel/JSON”; o prompt prevê apenas o caminho PDF → revisão → JSON e proíbe um sistema administrativo completo. Recomendação para o MVP: um arquivo JSON versionado por ano letivo/semestre, lido pelo CalendarioService; tela de revisão e leitura de PDF ficam para uma fase posterior.

9. Etapa de conferência e edição

É **obrigatória** antes da geração do DOCX final: o sistema gera o documento preenchido, o usuário visualiza, confere, pode editar (ex.: nome, matrícula, alguma disciplina, horário ou informação textual) e confirma antes de gerar a versão final.

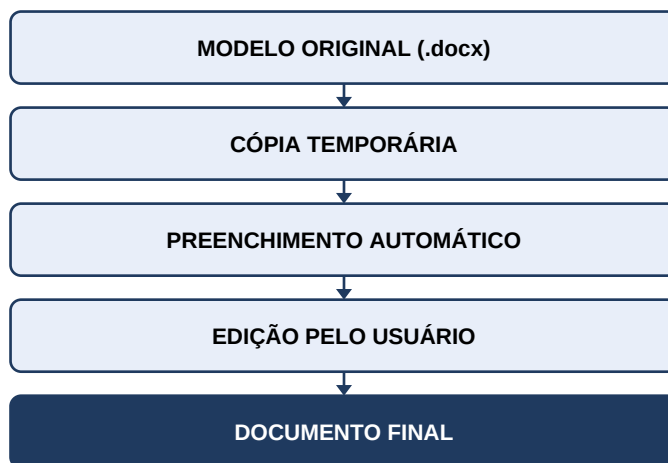


Figura 6 — A edição ocorre sobre uma cópia, preservando o modelo original

A edição nunca altera o modelo original. Também **não se assume** que toda edição feita pelo usuário precise ser gravada de volta no banco: o documento é o resultado gerado, e os dados oficiais continuam sendo os do cadastro existente.

9.1 Como a conferência pode funcionar tecnicamente

Opção	Como funciona	Vantagens	Desvantagens
A — Download e edição no Word	Sistema gera o DOCX preenchido; o usuário baixa, edita no Word e usa o arquivo como final.	Mais simples; usa a edição nativa do Word.	A “confirmação” e o “DOCX final” ficam fora do sistema; nenhuma validação após a edição.
B — Preview estruturado + regeneração (recomendada)	POST /preview devolve os dados da folha em JSON (campos editáveis). O usuário ajusta na tela e POST /gerar recebe os dados confirmados e gera o DOCX final.	Confirmação explícita; validação no servidor; sem estado no servidor; editável por campo.	Exige tela de conferência no frontend; edições de formatação livre não são possíveis.
C — Editor de DOCX embutido	Editor online (ex.: OnlyOffice/Collabora) dentro do sistema.	Edição completa e fiel ao Word.	Alto custo e dependência externa; fora do escopo confirmado.

DECISÃO A CONFIRMAR (D-11) — Estratégia de conferência/edição. Recomenda-se a opção B, mantendo A como complemento (link para baixar o DOCX de conferência). Campos editáveis sugeridos: nome, código, matrícula, regime, siglas das células da Grade Horária e observações textuais. Estrutura, fontes e bordas continuam controladas pelo template.

10. O que o sistema não implementa

O sistema não coloca automaticamente marcas de presença/falta (●, X, “PRESENTE”, “FALTA”) nem controla entrada, saída, atraso ou presença. O objetivo é preparar o documento, não controlá-lo.

- Controle de ponto, entrada, saída, presença, falta ou atraso.
- Justificativas de ausência.
- Reposição automática de aula ou substituição de professor.
- Controle de assinatura.
- Sistema paralelo de banco de dados; segundo cadastro de professores ou de grades.
- Sistema administrativo completo do Calendário Acadêmico.
- Login/autenticação e perfis de usuário, enquanto não definidos (**DECISÃO A CONFIRMAR (D-16)**).

Regra para quem implementa: não inventar funcionalidades, usuários, regras de negócio, campos de banco, regras de presença, reposição ou substituição. Se uma decisão for necessária, marcar “DECISÃO A CONFIRMAR” e levá-la aos responsáveis.

11. Arquitetura técnica

11.1 Tecnologias

Backend em **Java 21 + Spring Boot** (Maven), reutilizando o banco e as estruturas do projeto: Spring Web, Spring Data JPA, Hibernate, PostgreSQL, Jakarta Validation, **Apache POI** (*poi-ooxml*, para DOCX) e OpenAPI/Swagger (springdoc).

11.2 Organização de pacotes

```
src/main/java/br/com/fatec/backend/
|-- config/      configuracoes (Swagger, CORS, beans)
|-- controller/  endpoints REST (sem regra de negocio)
|-- service/     regras de negocio
|-- repository/  acesso ao banco (Spring Data JPA)
|-- entity/      entidades persistidas
|-- dto/         objetos de entrada e saida da API
|-- document/    manipulacao de DOCX (Apache POI)
|-- exception/   excecoes e tratamento global de erros
|-- util/        utilitarios puros (datas, textos)
-- BackendApplication.java
```

A Especificação lista *controller*, *service*, *repository*, *entity*, *dto*, *exception*, *util*; o prompt acrescenta o pacote *document*. A estrutura acima reúne os dois, isolando tudo o que depende de Apache POI em *document*.

11.3 Responsabilidade das camadas

Camada	Responsável por	Não deve
Controller	Receber a requisição, validar entrada básica, chamar o Service e devolver a Response.	Conter regra de negócio, acessar o banco ou manipular o DOCX.
Service	Regras de negócio e orquestração.	Conhecer detalhes de HTTP ou de células do DOCX.
Repository	Acesso ao banco.	Conter regra de negócio.
Entity	Representar o que é persistido.	Ser exposta diretamente pela API.
DTO	Dados que entram e saem da API.	Carregar lógica.
Document	Exclusivamente a manipulação de documentos (abrir template, preencher, sombrear, salvar).	Acessar banco ou decidir regras (recebe o modelo já pronto).
Exception	Tratamento centralizado de erros.	—

11.4 Services

Serviço	Pacote	Responsabilidade
ProfessorService	service	Busca e valida o professor; fornece nome, código, matrícula, regime e categoria.
GradeService	service	Busca a grade e as grades do professor; organiza as aulas; interpreta horários, SIGLA e converte para os slots do modelo; valida conflitos da grade.
CalendarioService	service	Consulta feriados, emendas, recessos e dias sem aula; informa se a data possui aula (baixa prioridade).
FolhaFrequenciaService	service	Coordena a geração: combina Professor + Grade + Mês/Ano + Calendário e monta o <i>FolhaModel</i> . Ex.: <i>gerarFolha(professorId, gradeId, mes, ano)</i> .
FolhaFrequenciaDocxService	document	Abre o template; localiza tabelas e células; substitui placeholders; preenche a grade; atualiza datas e dias da semana; aplica sombreadimento; salva o DOCX.
TemplateService	document	Carrega o template do <i>classpath</i> como somente leitura, entregando um <i>InputStream</i> novo a cada uso.

DECISÃO A CONFIRMAR (D-01) — Nomenclatura. A Especificação usa *FolhaFrequenciaService* / *DocxService*; o prompt usa *FolhaPontoService* / *FolhaPontoDocxService* e a rota */api/folhas-ponto*. Recomendação: usar “Frequência” (título oficial do documento), o que também evita confundir o módulo com controle de ponto. Este documento adota *FolhaFrequenciaService*, *FolhaFrequenciaDocxService* e */api/folhas-frequencia*.

11.5 Fluxo técnico

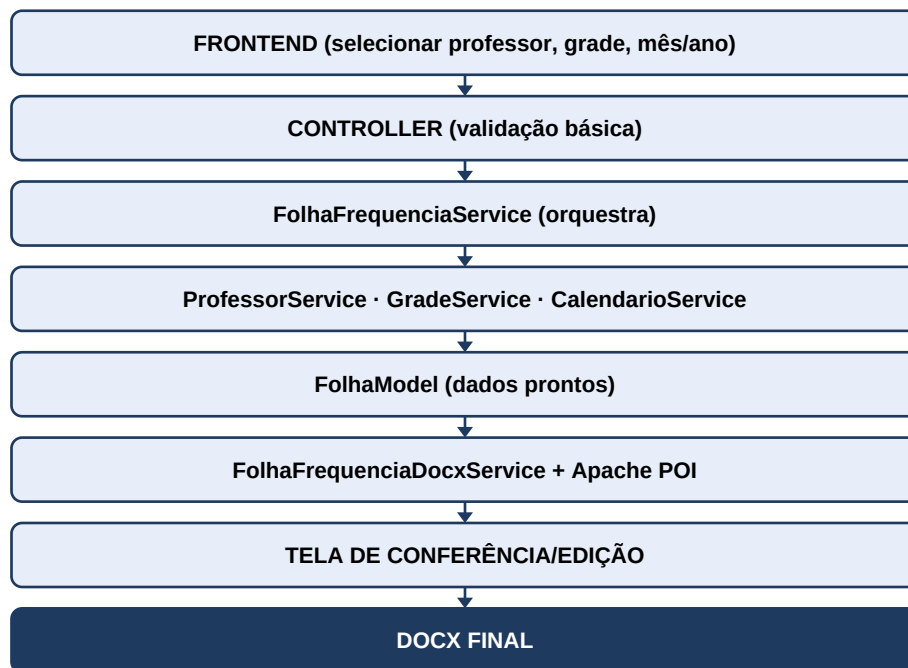


Figura 7 — Fluxo técnico completo

A lógica do calendário fica separada da geração do DOCX (`CalendarioService` → `FolhaFrequenciaService` → `FolhaFrequenciaDocxService`), para que mudanças futuras no calendário não exijam reescrever a lógica de geração. O `FolhaModel` (um *record*/DTO interno) é a fronteira entre negócio e documento: contém professor, mês/ano, células da Grade Horária (dia × slot → SIGLA) e a lista de dias (número, dia da semana, domingo, semAula, motivo). O `FolhaFrequenciaDocxService` recebe somente esse objeto — nunca acessa banco nem outros Services.

11.6 Exemplo de arquitetura (apenas esqueleto)

```

@RestController
@RequestMapping("/api/folhas-frequencia")
public class FolhaFrequenciaController {
    private final FolhaFrequenciaService service;

    public FolhaFrequenciaController(FolhaFrequenciaService service) { this.service = service; }

    @PostMapping("/preview")
    public ResponseEntity<FolhaPreviewResponse> preview(@Valid @RequestBody FolhaRequest req) {
        return ResponseEntity.ok(service.montarPreview(req)); // sem regra aqui
    }

    @PostMapping("/gerar")
    public ResponseEntity<byte[]> gerar(@Valid @RequestBody FolhaGerarRequest req) {
        byte[] docx = service.gerarDocumento(req);
        return ResponseEntity.ok()
            .contentType(MediaType.parseMediaType(
                "application/vnd.openxmlformats-officedocument.wordprocessingml.document"))
            .header(HttpHeaders.CONTENT_DISPOSITION, "attachment; filename=\"folha.docx\"")
            .body(docx);
    }
}
  
```

```

@Service
public class FolhaFrequenciaService {
    // injeta ProfessorService, GradeService, CalendarioService, FolhaFrequenciaDocxService

    public FolhaPreviewResponse montarPreview(FolhaRequest req) {
        // 1. professor 2. grade (pertence ao professor?) 3. converter horarios em slots
        // 4. dias do mes/domingos 5. eventos do calendario 6. montar FolhaModel (sem tocar no DOCX)
        return null;
    }

    public byte[] gerarDocumento(FolhaGerarRequest req) {
        FolhaModel modelo = /* dados confirmados + ajustes do usuario, revalidados */ null;
        return docxService.preencher(modelo);
    }
}

@Service
public class FolhaFrequenciaDocxService {
    public byte[] preencher(FolhaModel modelo) {
        // abre COPIA do template (somente leitura) -> substitui placeholders (runs) -> preenche grade
        // -> datas e dias da semana -> sombreamento -> limpa linhas inexistentes -> retorna bytes
        return null;
    }
}

```

11.7 Tratamento de erros

Um `@RestControllerAdvice` centraliza as respostas de erro (formato padrão *ProblemDetail*, RFC 7807), evitando `try/catch` nos Controllers.

Situação	Exceção sugerida	HTTP
Professor ou grade inexistente	RecursoNaoEncontradoException	404
Mês/ano inválido, campos obrigatórios ausentes	Bean Validation / RequisicaoInvalidaException	400
Grade não pertence ao professor; conflito de slots; SIGLA ausente	RegraNegocioException	422
Template ausente ou com estrutura inesperada	TemplateInvalidoException	500
Calendário indisponível	não bloqueia: gera aviso no preview	200 + aviso

11.8 Critérios de arquitetura

- Controllers simples; Services com as regras; Repositories só com o banco; DTOs para entrada/saída; Entities só para persistência; Document Services para o DOCX.
- Baixo acoplamento e alta coesão; evitar classes gigantes — dividir o Document Service em componentes (substituidor de placeholders, preenchedor da grade, preenchedor da tabela de datas, aplicador de sombreamento).
- Evitar lógica duplicada, regra de negócio no Controller, acesso ao banco no Controller e manipulação do DOCX no Controller.
- Injeção por construtor; configurações e caminhos de template em *application.properties*.

12. Rotas da API

Somente as rotas necessárias ao fluxo são propostas — **não** se cria CRUD completo. Os nomes dos campos de resposta dependem das entidades já existentes no Sistema de Grade e devem ser ajustados após a análise delas.

Método	URL	Objetivo	Service
GET	/api/professores	Listar professores para seleção	ProfessorService
GET	/api/professores/{id}	Dados do professor	ProfessorService
GET	/api/professores/{id}/grades	Grades disponíveis do professor	GradeService
GET	/api/grades/{id}	Grade com aulas (dia, horário, SIGLA, turma, curso)	GradeService
POST	/api/folhas-frequencia/preview	Gerar a prévia (dados da folha para conferência)	FolhaFrequenciaService
POST	/api/folhas-frequencia/gerar	Gerar o DOCX final com os dados confirmados	FolhaFrequenciaService

GET /api/professores

Objetivo	Listar professores para o usuário escolher.
Parâmetros	Opcional: <i>nome</i> (filtro por trecho do nome).
Request	—
Response 200	[{ "id": 12, "nome": "Alexandre Falcão Sanseverino" }]
Erros	500 (falha interna).
Service	ProfessorService

GET /api/professores/{id}

Objetivo	Obter os dados usados no bloco de identificação.
Parâmetros	<i>id</i> (path) — identificador do professor.
Response 200	{ "id": 12, "nome": "...", "codigo": null, "matricula": null, "regime": "Determinado", "categoria": "Professor" }
Erros	404 professor não encontrado.
Service	ProfessorService

GET /api/professores/{id}/grades

Objetivo	Listar as grades relacionadas ao professor, para seleção.
Parâmetros	<i>id</i> (path) — identificador do professor.
Response 200	[{ "id": 34, "descricao": "Grade 2026/2", "ano": 2026, "semestre": 2 }]
Erros	404 professor não encontrado.
Service	GradeService

GET /api/grades/{id}

Objetivo	Obter a grade com as aulas já organizadas por dia da semana e com o slot convertido.
Parâmetros	<i>id</i> (path) — identificador da grade.
Response 200	Lista de aulas: <i>diaSemana</i> , <i>horaInicio</i> , <i>horaFim</i> , <i>periodo</i> , <i>disciplina</i> { <i>codigo</i> , <i>sigla</i> , <i>nome</i> }, <i>turma</i> , <i>curso</i> , <i>slot</i> { <i>turno</i> , <i>ordem</i> }.
Erros	404 grade não encontrada.
Service	GradeService

POST /api/folhas-frequencia/preview	
Objetivo	Montar a prévia: reúne professor, grade convertida, datas do mês, domingos e dias sem aula, sem gravar nada. É a base da tela de conferência (seção 9).
Parâmetros	—
Request	{ "professorId": 12, "gradeId": 34, "mes": 9, "ano": 2026 } Validações: campos obrigatórios; mês 1–12; grade pertence ao professor.
Response 200	<i>FolhaPreviewResponse</i> (exemplo abaixo).
Erros	400 dados inválidos · 404 professor/grade inexistente · 422 regra de negócio (grade de outro professor, conflito de slots, SIGLA ausente).
Service	FolhaFrequenciaService (orquestra ProfessorService, GradeService, CalendarioService)

```
{
  "professor": { "nome": "Alexandre Falcao Sanseverino", "codigo": null, "matricula": null,
    "regime": "Determinado", "categoria": "Professor" },
  "mes": 9, "ano": 2026, "titulo": "FOLHA DE FREQUENCIA - 09 / 2026",
  "gradeHoraria": [ { "diaSemana": "SEGUNDA", "turno": "MANHA", "slot": 2, "sigla": "DW" } ],
  "dias": [ { "dia": 1, "diaSemana": "Terça-feira", "domingo": false, "semAula": false, "motivo": null },
    { "dia": 7, "diaSemana": "Segunda-feira", "domingo": false, "semAula": true,
      "motivo": "Independencia do Brasil" } ],
  "avisos": []
}
```

POST /api/folhas-frequencia/gerar	
Objetivo	Gerar o DOCX final a partir do modelo oficial, usando os dados confirmados (e eventuais ajustes feitos pelo usuário na conferência).
Parâmetros	—
Request	{ "professorId": 12, "gradeId": 34, "mes": 9, "ano": 2026, "ajustes": { ... } } <i>ajustes</i> é opcional e revalidado no servidor; seu formato final depende de DECISÃO A CONFIRMAR (D-11) .
Response 200	Arquivo DOCX (<i>application/vnd.openxmlformats-officedocument.wordprocessingml.document</i>) com <i>Content-Disposition: attachment; filename="Folha_Frequencia_{professor}_{MM}-{AAAA}.docx"</i> . Nada é gravado no servidor (DECISÃO A CONFIRMAR (D-15)).
Erros	400 · 404 · 422 (mesmos do preview) · 500 (template ausente/estrutura inesperada).
Service	FolhaFrequenciaService → FolhaFrequenciaDocxService

Futuro (baixa prioridade): uma rota de consulta de eventos do calendário (ex.: *GET /api/calendario/eventos?ano=&mes=*) só será proposta quando a importação/revisão do calendário for priorizada.

12.1 Swagger / OpenAPI e Insomnia

Todos os endpoints implementados devem estar documentados com OpenAPI (springdoc): *@Operation*, *@ApiResponse* para 200/400/404/422 e *@Schema* com exemplos nos DTOs. Swagger UI local: <http://localhost:8080/swagger-ui/index.html>. Uma coleção do Insomnia com uma requisição por rota, incluindo o download do DOCX, deve acompanhar o repositório.

13. Estrutura de pastas e templates

```
src/main/resources/
|-- templates/
|   |-- folha/
|   |   |-- folha-de-frequencia-oficial.docx      (modelo oficial - SOMENTE LEITURA)
|   |   |-- grade/
|   |       |-- grade-modelo.docx
|-- calendario/
|   |-- calendario-academico-2026-2.json          (baixa prioridade)
|-- application.properties                        (folha.template.path=templates/folha/...)

src/test/
|-- java/br/com/fatec/backend/                    (testes por camada)
|-- resources/templates/                          (copia do template usada nos testes)
```

- O modelo original é tratado como **somente leitura**: nunca aberto em modo de escrita e nunca salvo por cima.
- Leitura via *ClassPathResource* → *InputStream* (dentro do JAR não existe “arquivo” em disco); cada geração abre um stream novo.
- Se o *pom.xml* usar filtragem de recursos (*resource filtering*), excluir **.docx* dela — a filtragem corrompe arquivos binários.
- O prompt sugere o nome *folha-de-ponto-oficial.docx*; este documento usa *folha-de-frequencia-oficial.docx* (**DECISÃO A CONFIRMAR (D-01)**).

14. Estratégia de testes

Pirâmide: maioria de testes unitários (regras puras), testes de integração do DOCX e do banco, e testes de API. Ferramentas: Spring Boot Test, JUnit 5, Mockito, MockMvc, e Testcontainers (PostgreSQL) para repositórios quando aplicável.

Área	Tipo	O que testar
Busca de professor	Unit + integração	Professor existente e inexistente; mapeamento de nome, código, matrícula, regime, categoria.
Busca de grade	Unit + integração	Grades do professor; grade de outro professor (erro); aulas por dia da semana.
Conversão de horários	Unit (parametrizado)	Os 4 exemplos, empates, bordas e turnos (tabela abaixo).
Cálculo de datas	Unit	Setembro/2026, fevereiro/2027 (28), fevereiro/2028 (29), meses de 30 e 31 dias.
Identificação de domingos	Unit	Domingos de setembro/2026 (06, 13, 20, 27) e de fevereiro/2027 (07, 14, 21, 28).
Placeholders	Integração DOCX	Substituição de {{PROFESSOR}}, {{COD}}, {{MAT}}, {{RJ}}, {{MES}}, {{ANO}} — inclusive divididos em vários runs; nenhum “{” restante.
Preenchimento da Grade	Integração DOCX	SIGLA na célula correta (dia × slot); células não usadas vazias; conflito de slots.
Preenchimento da Folha	Integração DOCX	Número do dia, dia da semana e título; linhas 29–31 limpas em fevereiro.
Sombreamento	Integração DOCX	#D9D9D9 em todas as células das linhas de domingo; ausente nas demais.
Geração do DOCX	Integração	O arquivo abre com o Apache POI; estrutura (tabelas/linhas) idêntica à do modelo; fonte e bordas preservadas.
Preservação do template	Integração	Hash SHA-256 do modelo idêntico antes e depois da geração.
API	MockMvc / @WebMvcTest	200, 400, 404 e 422 de cada rota; /gerar devolve DOCX (assinatura “PK”) com Content-Disposition.
Calendário (baixa prioridade)	Unit	Evento com semAula = true; data sem evento; arquivo de calendário ausente gera aviso, não erro.

Casos críticos da conversão de horários (a regra de empate atravessa turnos):

Início real	Vizinhos	Esperado
07:30	07:30 exato	1º Manhã
08:00	07:30 (30) · 08:20 (20)	2º Manhã
08:50	08:20 (30) · 09:20 (30) — empate	3º Manhã
10:40	10:10 (30) · 11:10 (30) — empate	5º Manhã
12:30	12:00 (30) · 13:00 (30) — empate entre turnos	1º Tarde (13:00)
18:25	17:30 (55) · 19:20 (55) — empate entre turnos	1º Noite (19:20)
22:00	22:00 exato	4º Noite

15. Backlog e Issues

O backlog abaixo organiza o trabalho em 21 issues. As issues 1 a 18 formam o caminho principal; as issues 19 e 20 acompanham o desenvolvimento; a issue 21 (Calendário Acadêmico) é de **baixa prioridade**. Enquanto a issue 21 não existir, a issue 7 usa uma implementação do CalendarioService que devolve “nenhum evento”. O detalhamento de cada issue está no Anexo A.

#	Issue	Depende de	Prioridade
1	Estrutura inicial do backend	—	Alta
2	Integração com o banco existente	1	Alta
3	Mapeamento de Professor	2	Alta
4	Mapeamento de Grade	2	Alta
5	Implementação do GradeService	3, 4	Alta
6	Conversão de horários para slots	1	Alta
7	Implementação do FolhaFrequenciaService	5, 6	Alta
8	Template DOCX e mapa do modelo	1	Alta
9	Implementação do FolhaFrequenciaDocxService	8	Alta
10	Preenchimento do cabeçalho (professor)	3, 9	Alta
11	Preenchimento da Grade Horária	5, 6, 9	Alta
12	Atualização de mês e ano	9	Alta
13	Atualização das datas	7, 9	Alta
14	Atualização dos dias da semana	13	Alta
15	Tratamento dos domingos	13	Alta
16	Tratamento da formatação	15	Alta
17	Geração da prévia	7, 10–16	Alta
18	Geração do documento final	17	Alta
19	Testes automatizados	contínua	Média
20	Documentação Swagger	17, 18	Média
21	Calendário Acadêmico	7, 16	Baixa

15.1 Modelo de issue

Cada issue segue o modelo abaixo (o Anexo A o aplica de forma condensada):

```
# Titulo
## Descricao
## Objetivo
## Escopo
## Regras de negocio
## Arquivos/pastas envolvidos
## Services envolvidos
## Endpoints envolvidos
## Dependencias
## Testes
## Criterios de conclusao
## Observacoes
```

15.2 Critérios de conclusão globais

- Fluxo completo funcionando: professor → grade → mês/ano → prévia → confirmação → DOCX final.
- Modelo original inalterado (verificado por teste de hash) e nenhum “{” residual no documento.
- Domingos sombreados, dias e dias da semana corretos e linhas inexistentes limpas em qualquer mês.
- Grade Horária com SIGLA no slot correto pela regra de conversão.
- Nenhuma regra de negócio ou acesso ao banco nos Controllers; nenhum recurso de controle de ponto.
- Swagger com todos os endpoints; testes automatizados executando no build.

16. Checklist do escopo

Deve ser implementado:

- ✓ Selecionar professor, grade, mês e ano; buscar dados existentes.
- ✓ Preencher dados do professor e Grade Horária, utilizando a SIGLA.
- ✓ Converter horários reais para os slots do modelo.
- ✓ Atualizar mês, ano, datas e dias da semana; limpar dias inexistentes.
- ✓ Pintar domingos de cinza; consultar o Calendário Acadêmico e identificar dias sem aula (baixa prioridade).
- ✓ Gerar documento preenchido, permitir conferência/edição e gerar o DOCX final.

Não deve ser implementado nesta etapa:

- ✗ Controle de ponto, entrada, saída, presença, falta ou atraso.
- ✗ Justificativa, reposição automática ou substituição de professor.
- ✗ Controle de assinatura ou sistema paralelo de banco de dados.
- ✗ Sistema administrativo completo do Calendário Acadêmico.

17. Regra fundamental e resultado esperado

A implementação parte do que já existe no projeto: não cria estruturas paralelas, não cria um novo banco, não substitui o banco atual, não cria outro sistema de grade nem outro cadastro de professores.

Foco da etapa: automatizar a preparação da Folha de Frequência utilizando o banco existente, a Grade Horária e o Calendário Acadêmico; gerar o documento a partir do modelo oficial; e permitir que o usuário confira/edite antes de salvar o DOCX final — sem transformar a Folha de Frequência em um sistema de controle de presença/ponto.

Resultado esperado (exemplo de uso):

Passo	Ação
1	Selecionar: Alexandre Falcão Sanseverino
2	Selecionar: Grade 2026/2

Passo	Ação
3	Selecionar: Setembro / 2026 — clicar em Gerar Folha
4	Sistema busca dados, preenche a grade, atualiza datas e verifica o Calendário Acadêmico
5	Sistema gera o documento preenchido (prévia)
6	Usuário confere, edita e confirma
7	Sistema gera o DOCX final

18. Decisões pendentes e contradições

Fontes comparadas: a Especificação (PDF) e o prompt de requisitos. Cada item é uma **Contradição** (as fontes divergem) ou uma **Lacuna** (nenhuma fonte define). Este documento adotou a recomendação indicada apenas como hipótese de trabalho, até a confirmação dos responsáveis. Não estavam disponíveis para esta análise o arquivo DOCX oficial nem o código/entidades já existentes do Sistema de Grade.

ID	Situação	Impacto	Recomendação
D-01	Nomenclatura Frequência × Ponto [Contradição]. Especificação: FolhaFrequenciaService/DocxService. Prompt: FolhaPontoService/FolhaPontoDocxService, /api/folhas-ponto.	Nomes inconsistentes em código, API e Swagger; confusão com controle de ponto.	Usar "Frequência" em tudo: classes, rota e arquivo do template.
D-02	Prioridade do Calendário [Contradição]. Prompt: baixa prioridade, não bloqueante. Especificação: consta em "deve ser implementado".	Define se a primeira entrega já marca dias sem aula.	Criar CalendarioService com interface e retorno vazio; concluir por último (issue 21).
D-03	Fonte do calendário [Contradição]. Especificação: tela administrativa ou importação CSV/Excel/JSON. Prompt: PDF → revisão → JSON, sem sistema administrativo completo.	Esforço de tela e de armazenamento.	MVP: JSON versionado por semestre; tela de revisão e leitura de PDF em fase futura.
D-04	Semântica de semAula [Contradição interna]. Exemplo: FERIADO com semAula = false "(não há aula)".	Lógica invertida: feriados tratados como aula normal.	semAula = true significa sem aula; corrigir o exemplo (ou renomear para temAula).
D-05	Aparência do dia sem aula e precedência [Lacuna]. Nenhuma cor/texto definido; prompt cita "preenchimento definido pelo projeto".	Sem isso o sombreamento não pode ser implementado; conflito com domingo.	Cor distinta do cinza, sem texto; precedência: inexistente > domingo > sem aula > normal.
D-06	Escopo da regra do calendário [Lacuna]. Figura 4 só cruza datas com aula na grade; o prompt fala de dias sem aula em geral.	Quais linhas serão alteradas.	Seguir a Figura 4; listar todos os eventos do mês no preview.
D-07	Regra de conversão de horários [Lacuna]. Regra definida reproduz os 4 exemplos; faltam tolerância, aulas geminadas/duração e conferência com o código existente.	Aulas em slot errado na Grade Horária.	Validar contra o Sistema de Grade; avisar quando a distância passar de um limite (ex.: 30 min).
D-08	Exemplo "DW" nos 6 slots [Contradição]. Contraria a regra de um slot por aula.	Interpretação errada do preenchimento da grade.	Preencher só os slots resultantes da conversão; confirmar que o exemplo é ilustrativo.
D-09	Categoria do professor [Lacuna]. Aparece no exemplo e no ProfessorService; não há {{CATEGORIA}} no modelo.	Possível alteração de esquema sem necessidade.	Confirmar no modelo; adicionar a coluna apenas se o documento exibir a categoria.
D-10	Valores vazios (COD/MAT/RJ) [Lacuna]. Exemplo mostra "—".	Placeholder literal ou campo em branco no documento.	Substituir por "—"; permitir ajuste na conferência.
D-11	Conferência/edição [Lacuna]. A Especificação exige a etapa, mas não o mecanismo.	Define frontend, contrato de /gerar e esforço.	Opção B: preview estruturado + regeneração (seção 9.1).

ID	Situação	Impacto	Recomendação
D-12	Estrutura interna do DOCX [Lacuna]. Tabelas, linhas, células e placeholders reais não inspecionados.	Risco de índices incorretos e de quebrar o modelo.	Primeira tarefa da issue 8: mapa do template; centralizar índices em uma classe.
D-13	Mês fora do período da grade [Lacuna]. Ex.: Grade 2026/2 e mês 03/2026.	Folha vazia ou incoerente.	Avisar no preview, sem bloquear.
D-14	Conflito de slots [Lacuna]. Duas aulas convertidas para o mesmo dia e slot.	Perda de informação na célula.	Erro 422 descritivo (dia, slot e disciplinas).
D-15	Persistência do documento final [Lacuna]. "Gerar/salvar" não diz onde.	Armazenamento, histórico e privacidade.	Devolver para download, sem gravar no servidor ou no banco.
D-16	Autenticação e perfis [Lacuna]. Marcado como "caso ainda não definido".	Exposição de dados de professores.	Fora do escopo agora; tratar como requisito antes de expor a API fora da rede interna.

Anexo A — Issues detalhadas

Cada issue aplica o modelo da seção 15.1 de forma condensada. “Arquivos/pastas”, “Services” e “Endpoints” aparecem juntos; “Critérios de conclusão” e “Observações” também.

Issue 1 — Estrutura inicial do backend · Prioridade: Alta	
Descrição / Objetivo	Preparar a base do módulo em Java 21 + Spring Boot (Maven) com pacotes, dependências, Swagger e tratamento global de erros. Se o projeto de Grade já existir, apenas adicionar os pacotes/módulo — sem criar um projeto paralelo.
Escopo / Regras	Dependências: web, data-jpa, postgresql, validation, poi-ooxml, springdoc. Pacotes config, controller, service, repository, entity, dto, document, exception, util. @RestControllerAdvice base (ProblemDetail).
Arquivos · Services · Endpoints	pom.xml · BackendApplication.java · config/ · exception/GlobalExceptionHandler · Services: — · Endpoints: —
Dependências	—
Testes	Teste de contexto (contextLoads); Swagger UI acessível.
Conclusão / Observações	Build Maven verde, aplicação sobe e o Swagger abre. Obs.: respeitar a estrutura já existente do projeto.

Issue 2 — Integração com o banco existente · Prioridade: Alta	
Descrição / Objetivo	Conectar o módulo ao PostgreSQL já utilizado pelo Sistema de Grade, sem criar banco novo.
Escopo / Regras	Datasource por perfil; <i>ddl-auto=validate</i> (o Hibernate não cria nem altera tabelas); credenciais fora do código.
Arquivos · Services · Endpoints	application.properties (perfis) · config/ · Services: — · Endpoints: —
Dependências	Issue 1.
Testes	Teste de integração de conexão (Testcontainers ou perfil de teste).
Conclusão / Observações	Aplicação conecta ao banco atual e nenhuma tabela é criada automaticamente.

Issue 3 — Mapeamento de Professor · Prioridade: Alta	
Descrição / Objetivo	Reutilizar/ajustar a entidade de professor para expor nome, código, matrícula, regime (regime_contrato) e categoria.
Escopo / Regras	Verificar se cada campo existe; usar o existente; adicionar apenas o que faltar (D-09). Não duplicar o cadastro.
Arquivos · Services · Endpoints	entity/Professor · repository/ProfessorRepository · dto/ProfessorResponse · ProfessorService · GET /api/professores, GET /api/professores/{id}
Dependências	Issue 2.
Testes	Unit do ProfessorService; @DataJpaTest do repositório; MockMvc dos dois GET (200/404).
Conclusão / Observações	Rotas funcionando e documentadas; campos mapeados. Obs.: registrar impacto se o esquema mudar.

Issue 4 — Mapeamento de Grade · Prioridade: Alta

Descrição / Objetivo	Mapear as entidades existentes de grade, aula, disciplina (código, SIGLA, nome), turma e curso.
Escopo / Regras	Somente leitura; a Folha não cria cópia da grade. Necessário: dia da semana, horário inicial/final, período, SIGLA, turma, curso.
Arquivos · Services · Endpoints	entity/ (existentes) · repository/GradeRepository (busca por professor) · dto/GradeResponse, AulaResponse
Dependências	Issue 2.
Testes	@DataJpaTest com grade de exemplo (professor com mais de uma grade).
Conclusão / Observações	Grades e aulas lidas corretamente pelo repositório. Obs.: analisar o código existente antes de criar qualquer entidade.

Issue 5 — Implementação do GradeService · Prioridade: Alta

Descrição / Objetivo	Concentrar as regras da Grade: buscar grade, listar grades do professor, organizar aulas por dia da semana e validar conflitos.
Escopo / Regras	Validar que a grade pertence ao professor (422); detectar aulas no mesmo dia/slot (D-14); exigir SIGLA.
Arquivos · Services · Endpoints	service/GradeService · controller/GradeController, ProfessorController · GET /api/professores/{id}/grades, GET /api/grades/{id}
Dependências	Issues 3 e 4.
Testes	Unit com repositório simulado; MockMvc (200/404/422).
Conclusão / Observações	Rotas documentadas; regras cobertas por testes.

Issue 6 — Conversão de horários para slots · Prioridade: Alta

Descrição / Objetivo	Implementar a regra de conversão do horário real para o slot do modelo.
Escopo / Regras	Slot mais próximo; empate → próximo (posterior). Slots em enum único (07:30 ... 22:00). Avaliar tolerância e aulas geminadas (D-07). Fora do Controller.
Arquivos · Services · Endpoints	util ou service/ConversorHorarioSlot · SlotModelo (enum) · usado pelo GradeService
Dependências	Issue 1 (independe do banco).
Testes	Unit parametrizado: 08:00 → 2º, 08:50 → 3º, 09:50 → 4º, 10:40 → 5º, 12:30 → 1º tarde, 18:25 → 1º noite, 22:00 → 4º noite.
Conclusão / Observações	Todos os casos da seção 14 passando. Obs.: comparar com a regra existente no Sistema de Grade antes de alterar.

Issue 7 — Implementação do FolhaFrequenciaService · Prioridade: Alta

Descrição / Objetivo	Orquestrar a geração: professor + grade convertida + mês/ano + calendário → <i>FolhaModel</i> .
Escopo / Regras	Calcular dias do mês, dia da semana e domingos; consultar CalendarioService (stub vazio até a issue 21); produzir avisos (ex.: mês fora do período da grade, D-13).
Arquivos · Services · Endpoints	service/FolhaFrequenciaService · dto/FolhaModel, FolhaRequest · util/DiasDoMes · Services: Professor, Grade, Calendario
Dependências	Issues 5 e 6.
Testes	Unit com Services simulados; datas: set/2026, fev/2027, fev/2028.
Conclusão / Observações	FolhaModel completo e correto para qualquer mês. Obs.: sem regra de presença/falta.

Issue 8 — Template DOCX e mapa do modelo · Prioridade: Alta

Descrição / Objetivo	Incluir o modelo oficial no projeto e documentar sua estrutura interna (D-12).
Escopo / Regras	Template somente leitura; TemplateService entrega InputStream novo por uso; mapa do template: tabelas, linhas, colunas, células com placeholders e sombreamento; índices centralizados (TemplateLayout).
Arquivos · Services · Endpoints	resources/templates/folha/folha-de-frequencia-oficial.docx · document/TemplateService, TemplateLayout · documento “mapa do template”
Dependências	Issue 1.
Testes	Teste que abre o template e confere a estrutura esperada (nº de tabelas/linhas); hash do arquivo.
Conclusão / Observações	Mapa aprovado pelo Tech Lead. Obs.: excluir *.docx da filtragem de recursos do Maven.

Issue 9 — Implementação do FolhaFrequenciaDocxService · Prioridade: Alta

Descrição / Objetivo	Criar o serviço que abre uma cópia do template, aplica o FolhaModel e devolve os bytes do DOCX.
Escopo / Regras	Dividir em componentes: substituidor de placeholders, preenchedor da grade, preenchedor da tabela de datas, aplicador de sombreamento. Nunca acessar banco ou outros Services.
Arquivos · Services · Endpoints	document/FolhaFrequenciaDocxService e componentes · dto/FolhaModel
Dependências	Issue 8.
Testes	Gerar documento com FolhaModel mínimo; abrir com XWPFDocument; template intacto (hash).
Conclusão / Observações	Esqueleto gera um DOCX válido sem alterar o modelo.

Issue 10 — Preenchimento do cabeçalho (professor) · Prioridade: Alta

Descrição / Objetivo	Substituir {{PROFESSOR}}, {{COD}}, {{MAT}} e {{RJ}} (e a categoria, se existir no modelo — D-09).
Escopo / Regras	Tratar placeholders divididos em runs; preservar a formatação; valores vazios → “—” (D-10).
Arquivos · Services · Endpoints	document/PlaceholderReplacer · FolhaFrequenciaDocxService
Dependências	Issues 3 e 9.
Testes	Placeholder em um run e dividido em 3 runs; valor nulo; nenhum “{{” restante.
Conclusão / Observações	Bloco de identificação correto, com a formatação original.

Issue 11 — Preenchimento da Grade Horária · Prioridade: Alta

Descrição / Objetivo	Gravar a SIGLA no dia e slot corretos da Grade Horária do documento.
Escopo / Regras	Usar sempre a SIGLA (nunca o código); segunda a sábado; limpar células não usadas; um slot por aula (D-08).
Arquivos · Services · Endpoints	document/GradeHorariaFiller · GradeService (dados) · TemplateLayout
Dependências	Issues 5, 6 e 9.
Testes	Terça 08:00–08:50 DW → 2º da manhã; grade com várias aulas; nenhuma aula aos domingos.
Conclusão / Observações	Grade do documento reflete a grade selecionada.

Issue 12 — Atualização de mês e ano · Prioridade: Alta

Descrição / Objetivo	Preencher {{MES}} e {{ANO}} no título “FOLHA DE FREQUÊNCIA – MM / AAAA”.
Escopo / Regras	Mês com dois dígitos; ano com quatro.
Arquivos · Services · Endpoints	document/PlaceholderReplacer · FolhaFrequenciaDocxService
Dependências	Issue 9.
Testes	Título para 09/2026 e 02/2027.
Conclusão / Observações	Título correto em qualquer mês/ano.

Issue 13 — Atualização das datas · Prioridade: Alta

Descrição / Objetivo	Preencher o número do dia de cada linha e limpar as linhas dos dias inexistentes.
Escopo / Regras	01 até o último dia do mês; limpar 29–31 (fev. não bissexto), 30–31 (fev. bissexto) e 31 (meses de 30 dias); não criar nem remover linhas; nenhum dado antigo pode permanecer.
Arquivos · Services · Endpoints	document/FolhaTabelaFiller · util/DiasDoMes
Dependências	Issues 7 e 9.
Testes	Meses de 28, 29, 30 e 31 dias; linhas excedentes vazias.
Conclusão / Observações	Tabela correta em todos os tipos de mês.

Issue 14 — Atualização dos dias da semana · Prioridade: Alta

Descrição / Objetivo	Preencher o dia da semana de cada linha.
Escopo / Regras	Cálculo a partir da data; nomes por tabela fixa (sem depender de locale); grafia igual à do modelo.
Arquivos · Services · Endpoints	document/FolhaTabelaFiller · util/DiasDoMes
Dependências	Issue 13.
Testes	Set/2026: 01 Terça-feira ... 30 Quarta-feira; fev/2027 começa na segunda-feira.
Conclusão / Observações	Dias da semana corretos em qualquer mês.

Issue 15 — Tratamento dos domingos · Prioridade: Alta

Descrição / Objetivo	Sombrear de cinza (#D9D9D9) a linha inteira dos domingos.
Escopo / Regras	Aplicar o sombreamento (w:shd) em todas as células da linha; demais dias mantêm o preenchimento original.
Arquivos · Services · Endpoints	document/ShadingApplier
Dependências	Issue 13.
Testes	Domingos de set/2026 (06, 13, 20, 27) sombreados; outros dias não.
Conclusão / Observações	Todos os domingos sombreados, sem alterar borda, fonte ou altura.

Issue 16 — Tratamento da formatação · Prioridade: Alta

Descrição / Objetivo	Consolidar as regras visuais: precedência entre domingo, dia sem aula, dia inexistente e dia normal; garantir a preservação do modelo.
Escopo / Regras	Aparência do dia sem aula conforme D-05/D-06; linhas inexistentes sem sombreado; nenhuma linha criada; fonte, margens, bordas e altura preservadas.
Arquivos · Services · Endpoints	document/ShadingApplier · FolhaFrequenciaDocxService
Dependências	Issue 15.
Testes	Comparar propriedades (tcPr/rPr) das células não alteradas com o modelo; feriado em domingo.
Conclusão / Observações	Formatação idêntica ao modelo, exceto o que a regra manda alterar. Obs.: depende de D-05.

Issue 17 — Geração da prévia · Prioridade: Alta

Descrição / Objetivo	Expor o POST /preview com os dados da folha para a tela de conferência.
Escopo / Regras	Retornar FolhaPreviewResponse (professor, título, grade, dias, avisos); sem gravar nada. Definir o mecanismo de conferência (D-11) e, se adotado, o download do DOCX de conferência.
Arquivos · Services · Endpoints	controller/FolhaFrequenciaController · dto/FolhaPreviewResponse · POST /api/folhas-frequencia/preview
Dependências	Issues 7 e 10 a 16.
Testes	MockMvc: 200, 400, 404, 422; preview de set/2026 com domingos e avisos.
Conclusão / Observações	Prévia utilizável pelo frontend e documentada no Swagger.

Issue 18 — Geração do documento final · Prioridade: Alta

Descrição / Objetivo	Expor o POST /gerar, devolvendo o DOCX final com os dados confirmados.
Escopo / Regras	Revalidar dados e ajustes no servidor; nome do arquivo Folha_Frequencia_{professor}_{MM}-{AAAA}.docx; nada gravado no servidor (D-15).
Arquivos · Services · Endpoints	controller/FolhaFrequenciaController · dto/FolhaGerarRequest · POST /api/folhas-frequencia/gerar
Dependências	Issue 17.
Testes	MockMvc: content-type, Content-Disposition, assinatura "PK"; template inalterado após a chamada.
Conclusão / Observações	Arquivo baixa e abre no Word com a formatação oficial.

Issue 19 — Testes automatizados · Prioridade: Média

Descrição / Objetivo	Consolidar a estratégia de testes da seção 14 e integrá-la ao build.
Escopo / Regras	Unitários, integração DOCX, repositórios (Testcontainers) e API (MockMvc); regras críticas com cobertura mínima acordada.
Arquivos · Services · Endpoints	src/test/ · resources/templates (cópia para testes)
Dependências	Contínua (issues 1–18).
Testes	Toda a matriz da seção 14.
Conclusão / Observações	Testes rodando no build/CI. Obs.: cada issue já entrega seus testes.

Issue 20 — Documentação Swagger · Prioridade: Média

Descrição / Objetivo	Documentar todas as rotas com OpenAPI e manter uma coleção Insomnia.
Escopo / Regras	@Operation, @ApiResponse (200/400/404/422), @Schema com exemplos; coleção com uma requisição por rota (inclui o download do DOCX).
Arquivos · Services · Endpoints	config/OpenApiConfig · dto/ · coleção Insomnia no repositório
Dependências	Issues 17 e 18.
Testes	Verificação manual do Swagger UI (localhost:8080/swagger-ui/index.html).
Conclusão / Observações	Todos os endpoints implementados aparecem com exemplos.

Issue 21 — Calendário Acadêmico · Prioridade: BAIXA

Descrição / Objetivo	Implementar o CalendarioService com eventos (data, tipo, descrição, semAula) e integrar à Folha.
Escopo / Regras	Fonte inicial: JSON por semestre atrás de uma interface (D-03); semAula = true → sem aula (D-04); só datas com aula na grade (D-06); sem reposição, falta ou presença. Importação de PDF e tela de revisão: fase futura.
Arquivos · Services · Endpoints	service/CalendarioService · dto/EventoCalendario · resources/calendario/*.json
Dependências	Issues 7 e 16; decisões D-03 a D-06.
Testes	07/09/2026 (feriado, segunda com aula) marcado; 14/09/2026 normal; arquivo ausente gera aviso.
Conclusão / Observações	Dias sem aula refletidos no documento conforme a aparência aprovada. Obs.: não deve bloquear as demais issues.